



Solution Guide: Sysadmin Loops

In the previous activity, we created `for` loops. Now we will take our loops a bit further and use them to do useful things in our scripts and in the command line.

We can use loops to do things like:

- Loop through all the users on the system and take an action for each one.
- Loop through the results of a `find` command and take action on each item found.
- Loop through a list of log files and find files that contain a specific message.
- Loop through a group of files, check their permissions, and change them if needed.
- Loop through a group of files and create a cryptographic hash of each file.

In this activity, we created a few useful loops that we can add to our `sys_info.sh` script as well as loops you can use directly in the command line.

Solutions

Open the `sys_info.sh` script with `nano`.

1. Put the paths of the `shadow` and `passwd` files (from the `/etc` directory) in a list.

Solution:

```
Unset
files=(
  '/etc/passwd'
  '/etc/shadow'
)
```

Note: This can also be written in one line as follows:

```
Unset
files=(' /etc/passwd' ' /etc/shadow')
```

2. Create a `for` loop that prints out the permissions of each file in your file list.

Solution:

```
Unset
for file in ${files[@]}
do
ls -l $file >> $output
done
```

Syntax breakdown:

- `for file in ${files[@]}`: For each file in the list of `$files`.
- `do`: Complete the following command.
- `ls -l $file`: Run `ls -l` on each item in `$files`.
- `>> $output`: Write each output of `ls -l` to our output file.
- `done`: ends the `for` loop.

This could use a title:

- Type the following:

```
Unset
echo -e "\nThe permissions for sensitive /etc files: \n" >>
$output
for file in ${files[@]}
do
    ls -l $file >> $output
done
```

3. Add comments into our script.

- It is a best practice to add comments to explain the functionality in your scripts so that you and other developers can easily understand your code.

For example:

```
Unset
#Display CPU usage
echo -e "\nCPU Info:" >> $output
```

```

lscpu | grep CPU >> $output

# Display Disk usage
echo -e "\nDisk Usage:" >> $output
df -H | head -2 >> $output

#Display the current user
echo -e "\nCurrent user login information: \n $(who -a) \n" >>
$output

# ETC...

```

Bonus 1

Create a **for** loop that checks the **sudo** abilities of each user who has a home directory.

- `sudo -lU <username>` can be run on any user to see what `sudo` access they have.

Solution:

```

Unset
for user in $(ls /home)
do
    sudo -lU $user
done

```

Syntax breakdown:

- `for user in $(ls /home):` We use the `$()` command substitution directly in place of a list, because we know that the output `ls` is a list.
- `sudo -lU $user:` `sudo` checks for users in `/home`.
- `done` ends the `for` loop.

Run this command directly in the command line:

- **Solution:** `for user in $(ls /home); do sudo -lU $user; done`

The only difference to writing things on one line is the `;` used to separate each part of the loop.

Save and quit `nano`.

Bonus 2

Return to your script with `nano sys_info.sh`.

Create a list that contains the commands `date`, `uname -a`, and `hostname -s`.

Solution:

```
Unset
commands=(
    'date'
    'uname -a'
    'hostname -s'
)
```

Remove the lines that use these commands and replace them with a `for` loop that prints out, "The results of the _____ command are:" along with the results of running the command.

Solution:

```
Unset
for x in {0..2}
do
    results=$( ${commands[$x]} )
    echo "Results of \"${commands[$x]}\" command:"
    echo $results
    echo ""
done
```

Syntax breakdown:

- `for x in {0..2}`: Begins our `for` loop by looping through a list of numbers that serve as indices of our list.
 - We have 3 commands in our list. So the indices are 0, 1, and 2.
- `do`: Continues our `for` list.
- `results=$( )`: Assigns the output of each command to a temporary `results` variable.

- `${commands[$x]}`: The command name in the list at index `$x`, which resolves to 0, 1, or 2 depending on the iteration of the `for` loop.
- `echo "Results of \"${commands[$x]}\" command: ": For each iteration of the loop, we are printing 'Results of "${commands[x]}" command:'.

 - "${commands[x]}": The name of the command in our list with index $x. The output is then appended to our $output file.
 - echo $results >> $output: Prints the contents of the temporary $results variable to our $output file.
 - echo " ": Prints a new blank line.
 - done: ends our for loop.`

At this point, the `sys_info.sh` script should look similar to:

```
Unset
#!/bin/bash

#Check if script was run as root. Exit if false.
if [ $UID -ne 0 ]
then
    echo "Please do not run this script as root."
    exit
fi

# Define Variables
output=$HOME/research/sys_info.txt
ip=$(ip addr | grep inet | tail -2 | head -1)
execs=$(sudo find /home -type f -perm 777 2> /dev/null)
cpu=$(lscpu | grep CPU)
disk=$(df -H | head -2)

# Define Lists to use later
commands=(
    'date'
    'uname -a'
    'hostname -s'
)

files=(
    '/etc/passwd'
    '/etc/shadow'
)
```

```

#Check for research directory. Create it if needed.
if [ ! -d $HOME/research ]
then
    mkdir $HOME/research
fi

# Check for output file. Clear it if needed.
if [ -f $output ]
then
    > $output
fi

#####
#Start Script

echo "A Quick System Audit Script" >> $output
echo "" >> $output

for x in {0..2};
do
    results=$((${commands[$x]})
    echo "Results of \"${commands[$x]}\\" command:"
    echo $results
    echo ""

done

# Display Machine type
echo "Machine Type Info:" >> $output
echo -e "$MACHTYPE \n" >> $output

# Display IP Address info
echo -e "IP Info:" >> $output
echo -e "$ip \n" >> $output

# Display Memory usage
echo -e "\nMemory Info:" >> $output

```

```

free >> $output

#Display CPU usage
echo -e "\nCPU Info:" >> $output
lscpu | grep CPU >> $output

# Display Disk usage
echo -e "\nDisk Usage:" >> $output
df -H | head -2 >> $output

#Display login information for the current user
echo -e "\nCurrent user login information: \n $(who -a) \n" >>
$output

# Display DNS Info
echo "DNS Servers: " >> $output
cat /etc/resolv.conf >> $output

# List exec files
echo -e "\nexec Files:" >> $output
for exec in $execs;
do
    echo $exec >> $output
done

# List top 10 processes
echo -e "\nTop 10 Processes" >> $output
ps aux --sort -%mem | awk {'print $1, $2, $3, $4, $11'} | head >>
$output

# Check the permissions on files
echo -e "\nThe permissions for sensitive /etc files: \n" >> $output
for file in ${files[@]};
do
    ls -l $file >> $output
done

```

If you run the script, the contents of `sys_info.txt` should look similar to:

Unset

A Quick System Audit Script

Results of date command:

Mon Aug 26 17:12:59 EDT 2019

Results of uname -a command:

Linux work 4.14.119-2.pvops.qubes.x86_64 #1 SMP Wed May 15 06:43:11
UTC 2019 x86_64 GNU/Linux

Results of hostname -s command:

work

Machine Type Info:

x86_64-pc-linux-gnu

IP Info:

inet 10.137.0.15/32 brd 10.255.255.255 scope global eth0

Memory Info:

	total	used	free	shared	buff/cache
available					
Mem:	7956892	4368628	1226796	280532	2361468
3103552					
Swap:	1048572	15360	1033212		

CPU Info:

CPU op-mode(s): 32-bit, 64-bit
CPU(s): 4
On-line CPU(s) list: 0-3
CPU family: 6
Model name: Intel(R) Core(TM) i7-8650U CPU @ 1.90GHz
CPU MHz: 2112.068
NUMA node0 CPU(s): 0-3

Disk Usage:

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/xvda3	11G	9.1G	691M	93%	/

Current user login information:

	system boot	2019-08-23 13:02	
	run-level 3	2019-08-23 13:02	
LOGIN	hvc0	2019-08-23 13:02	681 id=hvc0
LOGIN	tty1	2019-08-23 13:02	683 id=tty1

exec Files:

DNS Servers:

nameserver 10.139.1.1

nameserver 10.139.1.2

Exec Files:

/home/sysadmin/Documents/setup_scripts/sysadmin/day3_stu_setup.sh
/home/instructor/Documents/setup_scripts/sysadmin/day3_stu_setup.sh
/home/instructor/Documents/setup_scripts/instructor/day3_setup.sh

Top 10 Processes

USER	PID	%CPU	%MEM	COMMAND
user	4997	0.3	4.6	/usr/lib/slack/slack
user	21470	0.5	4.5	/usr/lib/slack/slack
user	2618	0.8	3.8	/usr/share/atom/atom
user	8706	0.5	3.0	/opt/brave.com/brave/brave
user	1019	0.6	2.7	/opt/brave.com/brave/brave
user	2575	0.9	2.5	/usr/share/atom/atom
user	2909	0.7	2.5	/opt/zoom/zoom
user	8718	0.1	2.4	/opt/brave.com/brave/brave
user	3212	0.4	1.9	/usr/lib/slack/slack

The permissions for sensitive /etc files:

-rw-r--r--	1	root	root	1887	May 13 23:48	/etc/passwd
-rw-r-----	1	root	shadow	986	May 13 23:48	/etc/shadow